Viewpoint 1 — Corrected Statement

SCX Gauge Theory: What LayerNorm Does and Does Not Fix

SCX

July 2, 2026 / 202672

1 Original Claim (Incorrect) / ()

Original Viewpoint 1:

Experts live in their own coordinate systems — this is not a bug but gauge freedom.
Each expert output carries an unobservable offset g_m.
Training loss does not sense it — the residual connection absorbs it.
Routers / auditors / comparators do not know about it.
They compare things that are incomparable.

1:

——bug

g_m

——

//

2 Verdict /

Verdict: LARGELY INCORRECT /
The core intuition (parameter redundancy exists) has merit.
But every specific mechanism claim is wrong or requires heavy qualification.
LayerNorm eliminates constant offsets — **not** vector offsets.
Routers **can** detect gauge offsets.
Training loss **does** sense gauge offsets directly.

3 Corrected Viewpoint 1 / 1

Corrected Viewpoint 1:

Experts in a Mixture of Experts (MoE) operate in different coordinate systems: each expert m can carry an additive offset $\mathbf{g}_m \in \mathbb{R}^d$ in its output. This offset is a **gauge degree of freedom** — a parameter redundancy rather than a true observable.

However, the claim that these offsets are invisible to the training process, routers, and auditors is **incorrect**.

1. **LayerNorm:** In Pre-LN Transformers, LayerNorm eliminates the *per-token constant* component of $\mathbf{g}_m$ (all dimensions shifted by the same scalar) via mean subtraction. It does **not** eliminate *vector offsets* where different dimensions receive different shifts. The differential component $\mathbf{g}_m - \overline{\mathbf{g}_m} \cdot \mathbf{1}$ **leaks through** LayerNorm and propagates to downstream layers.
2. **Training loss:** The loss function directly senses $\mathbf{g}_m$. For MSE: $\mathcal{L}(y+\mathbf{g}_m, t) \neq \mathcal{L}(y, t)$. SGD compensates by adjusting learnable bias terms ($b \to b - \mathbf{g}_m$), which *is* sensing — the loss landscape changes, and optimization responds.
3. **Routers:** In Top-1 routing, $\mathbf{g}_m$ is **exactly recoverable** from observed outputs (error $< 10^{-15}$). In multi-layer MoE, $\mathbf{g}_m$ from layer l contaminates the input distribution of layer $l+1$, causing routing decisions to flip (empirically 5/8 tokens in a controlled experiment). Routers **do** know — their inputs are polluted.
4. **Auditors:** Given sufficient samples where expert m is activated, an auditor can recover $\mathbf{g}_m$ via ordinary least squares regression with error $< 10^{-14}$. The offset is statistically identifiable.

The real problem: It is not that gauge offsets are invisible. It is that existing architectures do not *explicitly* model the gauge degrees of freedom. Compensation happens **implicitly** — via SGD absorbing offsets into bias terms, and LayerNorm partially suppressing them. This implicit compensation degrades routing quality, pollutes inter-layer communication, and makes expert outputs incomparable without proper gauge fixing.

1:

MoE$m\mathbf{g}_m \in \mathbb{R}^d$ ——

1. **LayerNorm:** Pre-LN TransformerLayerNorm$\mathbf{g}_m$ *token* $\mathbf{g}_m - \overline{\mathbf{g}_m} \cdot \mathbf{1}$ LayerNorm
2. **:** $\mathbf{g}_m$ MSE: $\mathcal{L}(y + \mathbf{g}_m, t) \neq \mathcal{L}(y, t)$ SGD$b \to b - \mathbf{g}_m$——
3. **:** Top-1$\mathbf{g}_m < 10^{-15}$MoE$l\mathbf{g}_m l + 1$ 5/8token——
4. **:** m $\mathbf{g}_m < 10^{-14}$

: ——SGDLayerNorm

4 Precise Analysis: What LayerNorm Does and Does Not Do

5 : LayerNorm

5.1 Mathematics of LayerNorm Gauge Absorption

5.2 LayerNorm

Consider a Pre-LN Transformer block where the output of layer l passes through LayerNorm before entering layer $l+1$:

$$y^{(l)} = x^{(l)} + \mathrm{FFN}^{(l)}\big(\mathrm{LN}(x^{(l)})\big) \tag{1}$$

If expert m adds a gauge offset $\mathbf{g}_m$ to its output:

$$y^{(l)} = x^{(l)} + \mathrm{FFN}^{(l)}\big(\mathrm{LN}(x^{(l)})\big) + \mathbf{g}_m \tag{2}$$

The next layer's input is $\mathrm{LN}(y^{(l)})$. For a vector $z \in \mathbb{R}^d$:

$$\mathrm{LN}(z)_i = \frac{z_i - \mu(z)}{\sigma(z)} \cdot \gamma_i + \beta_i, \quad \mu(z) = \frac{1}{d}\sum_{j=1}^{d} z_j \tag{3}$$

Now consider two cases:

Case 1: Constant offset $\mathbf{g}_m = c \cdot \mathbf{1}$ (all dimensions shifted by the same scalar c).

$$\mu(y + c\mathbf{1}) = \mu(y) + c \tag{4}$$
$$y_i + c - \mu(y + c\mathbf{1}) = y_i + c - (\mu(y) + c) = y_i - \mu(y) \tag{5}$$
$$\sigma(y + c\mathbf{1}) = \sigma(y) \tag{6}$$

Therefore $\mathrm{LN}(y + c\mathbf{1}) = \mathrm{LN}(y)$. The constant offset is **completely eliminated**.

Case 2: Vector offset $\mathbf{g}_m$ where $\mathbf{g}_{m,i}$ varies across dimensions.

$$\mu(y + \mathbf{g}_m) = \mu(y) + \overline{\mathbf{g}_m} \tag{7}$$
$$(y_i + \mathbf{g}_{m,i}) - \mu(y + \mathbf{g}_m) = (y_i - \mu(y)) + (\mathbf{g}_{m,i} - \overline{\mathbf{g}_m}) \tag{8}$$
$$\sigma(y + \mathbf{g}_m) \neq \sigma(y) \quad \text{(in general)} \tag{9}$$

Only the mean component $\overline{\mathbf{g}_m}$ is subtracted. The **differential component $\mathbf{g}_{m,i} - \overline{\mathbf{g}_m}$** survives LayerNorm and leaks to the next layer.

1: $\mathbf{g}_m = c \cdot \mathbf{1}c$

$$\mu(y + c\mathbf{1}) = \mu(y) + c \tag{10}$$
$$y_i + c - \mu(y + c\mathbf{1}) = y_i + c - (\mu(y) + c) = y_i - \mu(y) \tag{11}$$
$$\sigma(y + c\mathbf{1}) = \sigma(y) \tag{12}$$

$\mathrm{LN}(y + c\mathbf{1}) = \mathrm{LN}(y)$

2: $\mathbf{g}_m\mathbf{g}_{m,i}$

$$\mu(y + \mathbf{g}_m) = \mu(y) + \overline{\mathbf{g}_m} \tag{13}$$

$$(y_i + \mathbf{g}_{m,i}) - \mu(y + \mathbf{g}_m) = (y_i - \mu(y)) + (\mathbf{g}_{m,i} - \overline{\mathbf{g}_m}) \tag{14}$$

$$\sigma(y + \mathbf{g}_m) \neq \sigma(y) \quad () \tag{15}$$

$\overline{\mathbf{g}_m}$ $\mathbf{g}_{m,i} - \overline{\mathbf{g}_m}$LayerNorm

5.3 Empirical Evidence /

Table 1: Gauge offset suppression by LayerNorm

Offset Type /	**Input Norm**	**LN Residual**	**Leaks?**
Constant $g = 3.0 \cdot \mathbf{1}$	12.00	$\sim 10^{-16}$	No /
Vector g (norm 7.63)	7.63	4.26	**Yes /**

Table 2: Summary of experimental evidence against original claims

Experiment /	**Result /**	**Refutes /**
Top-1 router g_m recovery	Exact (err $< 10^{-15}$)	"Router doesn't know"
LN vector offset test	Residual 4.26 leaks	"Residual absorbs"
Direct loss sensing	$\Delta\mathcal{L} = 7.79$	"Loss doesn't sense"
SGD bias absorption	$b_{\text{no } g} \approx b_{\text{with } g} + g$	Parameter redundancy ✓, not invisibility
Multi-layer route flip	5/8 tokens flip top-1	"Router doesn't know"
Statistical g_m separation	Exact (err $< 10^{-14}$)	"Auditor doesn't know"

6 Why Routers CAN Detect Gauge /

6.1 Top-1 Routing: Direct Observation

6.2 Top-1 :

In Top-1 routing, exactly one expert is active per token. The MoE output is:

$$y = \text{FFN}_{m^*}(x) + \mathbf{g}_{m^*} \tag{16}$$

The gauge offset $\mathbf{g}_{m^*}$ appears **directly** in the output. A downstream observer (or Layer $l+1$ router) sees y, which contains $\mathbf{g}_{m^*}$ as an additive term. If the observer also knows $\text{FFN}_{m^*}(x)$ (or can approximate it), $\mathbf{g}_{m^*}$ is trivially recoverable by subtraction.

Top-1tokenMoE:

$$y = \text{FFN}_{m^*}(x) + \mathbf{g}_{m^*} \tag{17}$$

$\mathbf{g}_{m^*} l + 1$ $y \mathbf{g}_{m^*} \text{FFN}_{m^*}(x)$ $\mathbf{g}_{m^*}$

6.3 Multi-Layer Contamination /

In a multi-layer MoE, the Layer l output feeds into Layer $l+1$'s router:

$$r^{(l+1)} = \text{Router}^{(l+1)}\big(y^{(l)}\big) = \text{Router}^{(l+1)}\big(x^{(l)} + \text{FFN}^{(l)}(\cdot) + \mathbf{g}_m\big) \tag{18}$$

The router computes $\alpha^{(l+1)}(x) = \text{softmax}(r^{(l+1)})$. Since $\mathbf{g}_m$ shifts $r^{(l+1)}$, the softmax weights change:

$$\alpha_k^{(l+1)} = \frac{\exp(r_k^{(l+1)} + \Delta_k)}{\sum_j \exp(r_j^{(l+1)} + \Delta_j)}, \quad \Delta = W_{\text{router}} \cdot \mathbf{g}_m \tag{19}$$

Consequence: $\mathbf{g}_m$ from layer l changes the routing decisions in layer $l+1$. This is direct evidence that routers **do** sense gauge offsets.

MoE$ll+1$:

$$r^{(l+1)} = \text{Router}^{(l+1)}\big(y^{(l)}\big) = \text{Router}^{(l+1)}\big(x^{(l)} + \text{FFN}^{(l)}(\cdot) + \mathbf{g}_m\big) \tag{20}$$

$\alpha^{(l+1)}(x) = \text{softmax}(r^{(l+1)})$ $\mathbf{g}_m r^{(l+1)}$softmax:

$$\alpha_k^{(l+1)} = \frac{\exp(r_k^{(l+1)} + \Delta_k)}{\sum_j \exp(r_j^{(l+1)} + \Delta_j)}, \quad \Delta = W_{\text{router}} \cdot \mathbf{g}_m \tag{21}$$

: $l\mathbf{g}_m l+1$

7 How the Training Process Senses Gauge /

The claim "training loss does not sense $\mathbf{g}_m$" conflates two distinct phenomena:

1. **Instantaneous loss:** $\mathcal{L}(y + \mathbf{g}_m, t) \neq \mathcal{L}(y, t)$ for any fixed $\mathbf{g}_m \neq 0$. The loss surface is **different**. Gradients $\partial\mathcal{L}/\partial\theta$ are shifted. Training **directly senses** $\mathbf{g}_m$.
2. **Post-convergence equivalence:** If the expert has a learnable bias b, then the effective bias after training is $b+\mathbf{g}_m$. SGD can reach the same optimal loss value (up to optimization noise) by setting $b^*_{\text{with } g} = b^*_{\text{without } g} - \mathbf{g}_m$. This is **compensation**, not insensitivity.

"$\mathbf{g}_m$":

1. **:** $\mathbf{g}_m \neq 0$ $\mathcal{L}(y + \mathbf{g}_m, t) \neq \mathcal{L}(y, t)$ $\partial\mathcal{L}/\partial\theta$ $\mathbf{g}_m$
2. **:** $bb + \mathbf{g}_m$ SGD$b^*_g = b^* - \mathbf{g}_m$

8 Comparison with SCX Gauge Theory / SCX

The MoE situation is **not** a true gauge symmetry in the physical sense. It is a **parameter redundancy** that the optimization process resolves implicitly. Unlike SCX where $\sum g_e = 0$ is an active human convention, MoE "gauge fixing" is a passive byproduct of gradient descent and normalization layers.

MoE SCX$\sum g_e = 0$ MoE""

Table 3: SCX gauge theory vs. MoE gauge redundancy

Aspect /	**SCX Gauge Theory**	**MoE Gauge Redundancy**
Gauge group /	$(\mathbb{R}, +)$ — single scalar shift	$(\mathbb{R}^d, +)^M$ — per-expert vector shift
Gauge fixing /	**Active**: $\sum g_e = 0$ by convention	**Passive**: SGD + LayerNorm choose implicitly
Observability /	True gauge invariance: physical observables unchanged	Pseudo-gauge: observables *are* affected, compensated post-hoc
LayerNorm role / LN	Not applicable	Eliminates constant component only; vector component leaks
Router awareness /	Not applicable	Directly affected; routing decisions change

9 Required Qualifications for Any "Gauge Freedom" Claim

10 ""

For the claim "experts operate in different coordinate systems due to gauge freedom" to be accurately stated, the following qualifications are necessary:

"":

1. **Learnable parameter condition / :** The expert must have learnable output biases capable of absorbing $\mathbf{g}_m$. Without learnable biases, $\mathbf{g}_m$ directly shifts the loss and is fully observable.

2. **Offset type condition / :** In Pre-LN architectures, only *per-token constant* offsets are eliminated by LayerNorm. *Vector offsets* (different values per dimension) leak through. The claim "residual connection absorbs" is wrong — it is LayerNorm that does (partial) absorption.

3. **Convergence condition / :** "Loss doesn't sense $\mathbf{g}_m$" is only true in the limit of perfect convergence where biases have fully absorbed the offset. During training, the loss surface differs and gradients are affected.

4. **Single-layer condition / :** "Router doesn't know" could only hold (weakly) for single-layer MoE where the router sees the original input x, not gauge-contaminated intermediate representations. In multi-layer MoE, Layer l offsets propagate to Layer $l + 1$ routing.

5. **Information-limiting condition / :** "Auditor doesn't know" requires the auditor to have insufficient samples to perform statistical separation. With adequate samples ($\gtrsim d$ per expert), $\mathbf{g}_m$ is recoverable via OLS regression with machine-precision accuracy.

11 Summary: The Correct Picture / :

The Correct Claim / :

Experts in MoE carry additive offsets $\mathbf{g}_m$ that constitute a form of parameter redundancy. In Pre-LN Transformers, LayerNorm eliminates the *constant* (per-token mean) component of these offsets, but *vector* components (dimension-specific differences) leak through the residual stream. Training senses these offsets directly and compensates via bias terms — this is active sensing, not ignorance. Routers in multi-layer MoE are directly affected: gauge offsets from one layer contaminate the input distribution of the next, causing routing decisions to flip. Auditors with sufficient samples can statistically recover the offsets with near-perfect accuracy.

The real problem is not invisibility — it is that existing architectures do not explicitly model the gauge degrees of freedom. Implicit compensation degrades routing quality and makes cross-expert comparisons unreliable. Proper gauge fixing (explicitly constraining offsets, as in SCX's $\sum g_e = 0$ convention) would make expert outputs comparable and improve routing stability.

MoE$\mathbf{g}_m$Pre-LN Transformer LayerNormtoken —— MoE

——

SCX$\sum g_e = 0$

Code Evidence /

All experimental claims are validated by `analysis_viewpoint1.py` (pure NumPy, no deep learning framework dependency). Key results:

`analysis_viewpoint1.py`NumPy:

- **PART 2:** Top-1 routing enables exact g_m recovery (error $< 10^{-15}$)
- **PART 3:** Constant offset eliminated by LN (residual $\sim 10^{-16}$); vector offset leaks (residual 4.26)
- **PART 4:** Direct loss sensing: $\Delta\mathcal{L} = 7.79$
- **PART 5:** SGD absorbs g into learnable bias (post-training difference norm 0.153)
- **PART 6:** Multi-layer contamination: 5/8 tokens flip top-1 routing
- **PART 7:** Statistical g_m separation via OLS: error $< 10^{-14}$